

Praca z danymi półstrukturalnymi

Dane półstrukturalne

Dane półstrukturalne to dane, które nie spełniają standardów tradycyjnych danych ustrukturyzowanych, ale zawierają tagi (etykiety) lub inne rodzaje oznaczeń, które identyfikują poszczególne, odrębne elementy w obrębie tych danych.

Dane półstrukturalne

Kluczowe cechy odróżniające dane częściowo ustrukturyzowane od danych ustrukturyzowanych to zagnieżdżone struktury danych oraz brak stałego schematu:

- Dane ustrukturyzowane wymagają stałego schematu, który należy zdefiniować przed załadowaniem danych i umożliwieniem ich przeszukiwania w relacyjnej bazie danych. Dane częściowo ustrukturyzowane **nie wymagają wcześniejszego zdefiniowania schematu i mogą podlegać ciągłym zmianom (tj. w dowolnym momencie można dodawać nowe atrybuty).**
- Ponadto obiekty należące do tej samej klasy mogą mieć różne atrybuty, mimo że są zgrupowane razem, a kolejność tych atrybutów nie ma znaczenia.

Dane półstrukturalne

- Obiekty należące do tej samej klasy mogą mieć różne atrybuty, mimo że są zgrupowane razem, a kolejność tych atrybutów nie ma znaczenia
- W przeciwieństwie do danych ustrukturyzowanych, które przedstawiają dane w postaci płaskiej tabeli, dane częściowo ustrukturyzowane mogą zawierać wielopoziomowe hierarchie zagnieżdżonych informacji.
- Snowflake obsługuje JSON, Avro, ORC, Parquet i XML oraz pozwala ładować je bez pełnego, sztywnego schematu wejściowego

Struktura hierarchiczna

- Dane częściowo ustrukturyzowane są zazwyczaj zorganizowane hierarchicznie. Złożone struktury danych można tworzyć poprzez zagnieżdżanie prostszych typów danych, takich jak tablice i obiekty. (Obiekt Snowflake nie jest „obiektem” w rozumieniu „programowania obiektowego”.)
- Na przykład dane JSON mogą zawierać obiekt zawierający tablicę. Każda komórka tej tablicy może sama w sobie zawierać zagnieżdżony obiekt lub tablicę.

Struktura hierarchiczna

- Można wykorzystać typy danych Snowflake do utworzenia hierarchii służącej do przechowywania danych częściowo ustrukturyzowanych, korzystając z następujących właściwości typów danych:
 - VARIANT może przechowywać wartość dowolnego innego typu danych, w tym tablicę (ARRAY) lub obiekt (OBJECT).
 - Typ ARRAY lub OBJECT przechowuje wartość typu VARIANT.

Struktura hierarchiczna

- Załóżmy na przykład, że chcesz zapisać daty, w których miały miejsce różne rodzaje klęsk żywiołowych.
- Możesz utworzyć obiekt zawierający klucze „Huragan”, „Trzęsienie ziemi”, „Powódź” itp.
- Wartością powiązaną z każdym z tych kluczy może być ARRAY zawierająca daty wystąpienia każdego rodzaju katastrofy.
- Ponieważ wartość w każdej parze klucz-wartość musi być typu VARIANT, każda tablica dat byłaby przechowywana jako ARRAY zawarta w obiekcie typu VARIANT wewnątrz odpowiedniego OBIEKTU.
- Najwyższy poziom hierarchii wyglądałby podobnie do poniższego (nawiasy klamrowe oznaczają OBIEKT, który zawiera pary klucz-wartość):

```
{  
  "Flood": flood_date_array::VARIANT,  
  "Earthquake": earthquake_date_array::VARIANT,  
  ...  
}
```

Struktura hierarchiczna

- Jako kolejny przykład założmy, że chcesz zapisać pojedynczą listę katastrof w porządku chronologicznym.
- W takim przypadku zewnętrznym typem danych może być ARRAY.
- Każda komórka tego ARRAY może zawierać OBJECT (opakowany w VARIANT), który zawiera pary klucz-wartość z informacjami o zdarzeniu.
- Na przykład każdy OBJECT opisujący trzęsienie ziemi może mieć klucze takie jak „Timestamp”, „Location” i „Magnitude”.
- Każdy OBJECT opisujący tornado może mieć klucze takie jak „Timestamp” i „Maximum_wind_speed”.

```
[  
  {  
    "Event_ID": 54::VARIANT,  
    "Type": "Earthquake"::VARIANT,  
    "Magnitude": 7.4::VARIANT,  
    "Timestamp": "2018-06-09 12:32:15"::TIMESTAMP_LTZ::VARIANT  
    ...  
  }::VARIANT,  
  {  
    "Event_ID": 55::VARIANT,  
    "Type": "Tornado"::VARIANT,  
    "Maximum_wind_speed": 186::VARIANT,  
    "Timestamp": "2018-07-01 09:42:55"::TIMESTAMP_LTZ::VARIANT  
    ...  
  }::VARIANT  
]
```


Struktura hierarchiczna

- Można tworzyć hierarchie danych o niemal dowolnej głębokości lub rozpiętości (w granicach możliwości pamięci dla każdego typu danych).
- Na przykład obiekt (OBJECT) zawierający informacje o tornadzie może wymagać danych dotyczących prędkości wiatru w różnych momentach trwania tornada, więc struktura danych może wyglądać następująco:
 - Najwyższy poziom stanowi ARRAY
 - Każda komórka ARRAY zawiera jeden OBIEKT opisujący jedno tornado.
 - Każdy OBIEKT zawiera ARRAY danych dotyczących prędkości wiatru.
 - Każda komórka tej wewnętrznej ARRAY jest OBIEKTEM zawierającym dane z następującymi kluczami:
 - Sygnatura czasowa pomiaru prędkości wiatru.
 - Lokalizacja pomiaru prędkości wiatru.
 - Prędkość wiatru w km/h (kilometrach na godzinę).
- W niektórych przypadkach dane mogą być niekompletne. Na przykład, jeśli prędkość wiatru w danej lokalizacji oszacowano na podstawie widocznych zniszczeń po przejściu tornada (a nie zmierzono bezpośrednio w trakcie jego trwania), dane mogą zawierać lokalizację i prędkość wiatru, ale nie będą zawierały daty i godziny.

Przechowywanie w typie VARIANT

- Wartość typu VARIANT może przechowywać wartości dowolnego innego typu, w tym wartości typu OBJECT i ARRAY.
- Wartość typu VARIANT może mieć maksymalny rozmiar wynoszący do 128 MB nieskompresowanych danych. Jednak w praktyce maksymalny rozmiar jest zazwyczaj mniejszy ze względu na wewnętrzne obciążenie systemowe. Maksymalny rozmiar zależy również od przechowywanego obiektu.
- Aby bezpośrednio wstawić dane typu VARIANT, należy użyć instrukcji INSERT INTO ... SELECT. Poniższy przykład pokazuje, jak wstawić dane w formacie JSON do wartości typu VARIANT:

```
CREATE OR REPLACE TABLE variant_insert (v VARIANT);
INSERT INTO variant_insert (v)
  SELECT PARSE_JSON('{"key3": "value3", "key4": "value4"}');
SELECT * FROM variant_insert;
```

```
+-----+
| V      |
+-----+
| {      |
|  "key3": "value3", |
|  "key4": "value4"  |
| }      |
+-----+
```

Przechowywanie w typie VARIANT

- Aby przekonwertować wartość na typ danych VARIANT lub z tego typu, można użyć jawnego rzutowania za pomocą funkcji CAST, funkcji TO_VARIANT lub operatora :: (na przykład wyrażenie::VARIANT).
- W niektórych sytuacjach wartość może zostać niejawnie rzutowana na wartość typu VARIANT.
- Poniższy przykład pokazuje, jak używać wartości typu VARIANT, w tym jak konwertować z wartości typu VARIANT i na wartość typu VARIANT.

```
CREATE OR REPLACE TABLE varia (float1 FLOAT, v VARIANT, float2 FLOAT);  
INSERT INTO varia (float1, v, float2) VALUES (1.23, NULL, NULL);
```

```
UPDATE varia SET v = TO_VARIANT(float1); -- converts from a FLOAT value to a VARIANT value.  
UPDATE varia SET float2 = v::FLOAT;      -- converts from a VARIANT value to a FLOAT value.
```

```
SELECT * FROM varia;
```

```
+-----+-----+-----+  
| FLOAT1 | V                | FLOAT2 |  
+-----+-----+-----+  
| 1.23   | 1.230000000000000e+00 | 1.23   |  
+-----+-----+-----+
```

Przechowywanie w typie VARIANT

- Domyślnie podczas pobierania wartości typu VARCHAR, DATE, TIME i TIMESTAMP z kolumny typu VARIANT wartości te są ujęte w cudzysłowy. Można usunąć cudzysłowy, jawnie rzutując wartości na typy danych bazowe (na przykład z typu VARIANT na VARCHAR). Na przykład:

```
SELECT 'Sample', 'Sample'::VARIANT, 'Sample'::VARIANT::VARCHAR;
```

'SAMPLE'	'SAMPLE'::VARIANT	'SAMPLE'::VARIANT::VARCHAR
Sample	"Sample"	Sample

Przechowywanie w typie VARIANT

- Wartość typu VARIANT może być pominięta (zawierać wartość SQL NULL), co różni się od wartości null typu VARIANT, która jest rzeczywistą wartością służącą do reprezentowania wartości null w danych częściowo ustrukturyzowanych.
- Wartość null typu VARIANT jest wartością logiczną, która w porównaniu jest równa sobie samej.
- Jeśli dane zostały załadowane w formacie JSON i zapisane w kolumnie typu VARIANT, należy wziąć pod uwagę następujące kwestie:
 - W przypadku danych, które są w większości standardowe i wykorzystują wyłącznie natywne typy JSON (takie jak ciągi znaków i liczby), wydajność operacji związanych z przechowywaniem i wyszukiwaniem danych relacyjnych oraz danych w kolumnie typu VARIANT jest bardzo zbliżona.
 - W przypadku danych nienatycznych (takich jak daty i znaczniki czasu) wartości są przechowywane jako ciągi znaków po załadowaniu do kolumny typu VARIANT. W związku z tym operacje na tych wartościach mogą przebiegać wolniej, a także zajmować więcej miejsca niż w przypadku przechowywania w kolumnie relacyjnej o odpowiednim typie danych.

Wartość NULL

- Snowflake obsługuje dwa rodzaje wartości NULL w danych częściowo ustrukturyzowanych:
 - SQL NULL: Wartość SQL NULL ma takie samo znaczenie dla typów danych częściowo ustrukturyzowanych, jak dla typów danych ustrukturyzowanych: wartość jest nieobecna lub nieznana.
 - JSON null (czasami nazywane „VARIANT NULL”): W kolumnie typu VARIANT wartości JSON null są przechowywane jako ciąg znaków zawierający słowo „null”, aby odróżnić je od wartości SQL NULL.
- Poniższy przykład porównuje wartości SQL NULL i JSON null:

```
SELECT PARSE_JSON(NULL) AS "SQL NULL",  
       PARSE_JSON('null') AS "JSON NULL",  
       PARSE_JSON('[ null ]') AS "JSON NULL",  
       PARSE_JSON('{ "a": null }'):a AS "JSON NULL",  
       PARSE_JSON('{ "a": null }'):b AS "ABSENT VALUE";
```

SQL NULL	JSON NULL	JSON NULL	JSON NULL	ABSENT VALUE
NULL	null	[null]	null	NULL
		null		
		]		

Wartość NULL

- Aby przekonwertować wartość null w formacie JSON na wartość NULL w języku SQL, należy ją rzutować na typ ciągu znaków. Na przykład:

```
SELECT PARSE_JSON('{ "a": null }'):a,  
       TO_CHAR(PARSE_JSON('{ "a": null }'):a);
```

```
+-----+-----+  
| PARSE_JSON('{ "A": NULL }'):A | TO_CHAR(PARSE_JSON('{ "A": NULL }'):A) |  
+-----+-----+  
| null                          | NULL                                |  
+-----+-----+
```

- Podczas tworzenia wartości typu ARRAY można określić elementy tablicy zawierające wartości NULL w języku SQL lub wartości null w formacie JSON. Poniższy przykład przedstawia utworzenie tablicy zawierającej oba rodzaje wartości NULL:

```
SELECT ARRAY_CONSTRUCT(1, NULL, PARSE_JSON('null')) AS array_with_null_values;
```

```
+-----+  
| ARRAY_WITH_NULL_VALUES |  
+-----+  
| [  
|   1,  
|   undefined,  
|   null  
| ]  
+-----+
```

Typ OBJECT

- Wartość typu OBJECT w Snowflake odpowiada „obiekтови” w formacie JSON.
- Wartość typu OBJECT zawiera pary klucz-wartość.
- W półustrukturyzowanych danych typu OBJECT w Snowflake każdy klucz jest wartością typu VARCHAR, a każda wartość jest wartością typu VARIANT.
- Ponieważ wartość typu VARIANT może przechowywać wartość dowolnego innego typu danych, różne wartości typu VARIANT (w różnych parach klucz-wartość) mogą mieć różne typy danych bazowych. Na przykład wartość typu OBJECT może przechowywać imię i nazwisko osoby jako wartość typu VARCHAR, a wiek tej osoby jako wartość typu INTEGER. W poniższym przykładzie zarówno imię i nazwisko, jak i wiek są rzutowane na wartości typu VARIANT.

```
SELECT OBJECT_CONSTRUCT(  
  'name', 'Jones'::VARIANT,  
  'age', 42::VARIANT);
```


Typ OBJECT

- W odniesieniu do danych typu OBJECT obowiązują następujące zasady:
 - Obecnie Snowflake nie obsługuje obiektów z jawnym typowaniem.
 - W parze klucz-wartość klucz nie może być pustym ciągiem znaków, a ani klucz, ani wartość nie mogą mieć wartości NULL.
 - Maksymalna długość wartości typu OBJECT wynosi 128 MB.
 - Wartość typu OBJECT może zawierać dane częściowo ustrukturyzowane.
 - Wartość typu OBJECT może służyć do tworzenia hierarchicznych struktur danych.

Typ OBJECT

- Aby wstawić dane typu OBJECT bezpośrednio, należy użyć instrukcji INSERT INTO ... SELECT.
- W poniższym przykładzie wykorzystano funkcję OBJECT_CONSTRUCT do utworzenia wartości typu OBJECT, która jest następnie wstawiana.

```
CREATE OR REPLACE TABLE object_example (object_column OBJECT);
INSERT INTO object_example (object_column)
  SELECT OBJECT_CONSTRUCT('thirteen', 13::VARIANT, 'zero', 0::VARIANT);
SELECT * FROM object_example;
```

```
+-----+
| OBJECT_COLUMN |
+-----+
| {             |
|   "thirteen": 13, |
|   "zero": 0      |
| }             |
+-----+
```

- W każdej parze klucz-wartość wartość była jawnie rzutowana na typ VARIANT.
- W tych przypadkach jawne rzutowanie nie było konieczne. Snowflake może dokonywać niejawnego rzutowania na typ VARIANT.
- Można również użyć stałej typu OBJECT, aby określić wartość typu OBJECT do wstawienia

Typ OBJECT

- OBJECT constants

- Stała oznacza niezmienną wartość danych.
- Snowflake obsługuje używanie stałych do określania wartości typu OBJECT.
- Stałe typu OBJECT są ujęte w nawiasy klamrowe ({ i }).
- Stałe typu OBJECT mają następującą składnię:

```
{ [<key>: <value> [, <key>: <value> , ...]] }
```

- Key - klucz w parze klucz-wartość. Klucz musi być literałem łańcuchowym.
- Value - wartość powiązana z kluczem. Wartość ta może być literałem lub wyrażeniem. Może ona mieć dowolny typ danych.

Typ OBJECT

- przykłady określające stałe typu OBJECT:
 - `{}` to pusta wartość typu OBJECT.
 - `{'key1': 'value1', 'key2': 'value2'}` zawiera określone pary klucz-wartość dla wartości typu OBJECT, przy czym jako wartości użyto literałów.
 - `{'key1': c1+1, 'key2': c1+2}` zawiera określone pary klucz-wartość dla wartości OBJECT, wykorzystując wyrażenia jako wartości.
 - `{*}` jest symbolem wieloznacznym, który tworzy wartość OBJECT na podstawie określonych danych, wykorzystując nazwy atrybutów jako klucze, a powiązane z nimi wartości jako wartości.

Typ OBJECT

- Jeśli symbol wieloznaczny występuje w stałej obiektowej, może być użyty w postaci nieokreślonej lub w połączeniu z nazwą tabeli lub aliasem. Na przykład obie poniższe specyfikacje symboli wieloznacznych są poprawne:

```
SELECT {*} FROM my_table;
```

```
SELECT {my_table1.*}  
FROM my_table1 INNER JOIN my_table2  
ON my_table2.col1 = my_table1.col1;
```

Typ OBJECT

- W stałej obiektowej można używać słów kluczowych ILIKE i EXCLUDE. Aby wybrać określone kolumny, należy użyć słowa kluczowego ILIKE. Na przykład poniższe zapytanie wybiera kolumny pasujące do wzorca col1% w tabeli my_table:

```
SELECT {*} ILIKE 'col1%'} FROM my_table;
```

- Aby wykluczyć określone kolumny, należy użyć słowa kluczowego EXCLUDE. Na przykład poniższe zapytanie wyklucza kolumnę col1 z tabeli my_table:

```
SELECT {*} EXCLUDE col1} FROM my_table;
```

```
SELECT {*} EXCLUDE (col1, col2)} FROM my_table;
```

Typ OBJECT

- W poniższych instrukcjach wykorzystano stałą OBJECT oraz funkcję OBJECT_CONSTRUCT w celu wstawienia danych typu OBJECT do tabeli. Wartości typu OBJECT zawierają nazwy i stolice dwóch prowincji kanadyjskich.

```
CREATE OR REPLACE TABLE my_object_table (my_object OBJECT);

INSERT INTO my_object_table (my_object)
  SELECT { 'PROVINCE': 'Alberta':VARIANT , 'CAPITAL': 'Edmonton':VARIANT };

INSERT INTO my_object_table (my_object)
  SELECT OBJECT_CONSTRUCT('PROVINCE', 'Manitoba':VARIANT , 'CAPITAL', 'Winnipeg':VARIANT )

SELECT * FROM my_object_table;
```

```
+-----+
| MY_OBJECT |
+-----+
| {         |
|   "CAPITAL": "Edmonton", |
|   "PROVINCE": "Alberta" |
| }         |
| {         |
|   "CAPITAL": "Winnipeg", |
|   "PROVINCE": "Manitoba" |
| }         |
+-----+
```

Typ OBJECT

- W poniższym przykładzie wykorzystano symbol wieloznaczny ({*}) do wstawienia danych typu OBJECT poprzez pobranie nazw atrybutów i wartości z klauzuli FROM. Najpierw należy utworzyć tabelę o nazwie **demo_ca_provinces** zawierającą wartości typu **VARCHAR** z nazwami prowincji i stolic:

```
CREATE OR REPLACE TABLE demo_ca_provinces (province VARCHAR, capital VARCHAR);
INSERT INTO demo_ca_provinces (province, capital) VALUES
  ('Ontario', 'Toronto'),
  ('British Columbia', 'Victoria');

SELECT province, capital
FROM demo_ca_provinces
ORDER BY province;
```

PROVINCE	CAPITAL
British Columbia	Victoria
Ontario	Toronto

```
INSERT INTO my_object_table (my_object)
| SELECT {*} FROM demo_ca_provinces;

SELECT * FROM my_object_table;
```

MY_OBJECT
{
"CAPITAL": "Edmonton",
"PROVINCE": "Alberta"
}
{
"CAPITAL": "Winnipeg",
"PROVINCE": "Manitoba"
}
{
"CAPITAL": "Toronto",
"PROVINCE": "Ontario"
}
{
"CAPITAL": "Victoria",
"PROVINCE": "British Columbia"
}

Typ OBJECT

- W poniższym przykładzie wykorzystano wyrażenia do określenia wartości w stałej typu OBJECT:

```
SET my_variable = 10;  
SELECT {'key1': $my_variable+1, 'key2': $my_variable+2};
```

```
+-----+  
| {'KEY1': $MY_VARIABLE+1, 'KEY2': $MY_VARIABLE+2} |  
+-----+  
| {  
|   "key1": 11,  
|   "key2": 12  
| }  
+-----+
```

- W instrukcjach SQL literały łańcuchowe wewnątrz wartości typu OBJECT są zapisywane w pojedynczych cudzysłowach (podobnie jak w pozostałych częściach języka SQL w Snowflake), jednak w wyświetlaniu wartości typu OBJECT literały te są przedstawiane w podwójnych cudzysłowach:

```
SELECT { 'Manitoba': 'Winnipeg' } AS province_capital;
```

```
+-----+  
| PROVINCE_CAPITAL |  
+-----+  
| {  
|   "Manitoba": "Winnipeg"  
| }  
+-----+
```

Typ OBJECT

- Aby pobrać wartość z obiektu OBJECT, należy podać klucz w nawiasach kwadratowych lub użyć operatora dwukropka.:

```
SELECT object_column['thirteen'],  
       object_column:thirteen  
FROM object_example;
```

OBJECT_COLUMN['THIRTEEN']	OBJECT_COLUMN:THIRTEEN
13	13

Typ OBJECT

- Dane typu OBJECT są zazwyczaj wykorzystywane, gdy spełniony jest co najmniej jeden z poniższych warunków:
 - Dysponujesz wieloma elementami danych identyfikowanymi za pomocą ciągów znaków. Na przykład, jeśli chcesz wyszukiwać informacje według nazwy województwa, warto użyć wartości typu OBJECT.
 - Chcesz przechowywać informacje o danych wraz z samymi danymi. Nazwy (klucze) nie są jedynie unikalnymi identyfikatorami, ale mają konkretne znaczenie.
 - Informacje nie mają naturalnej kolejności lub kolejność można wywnioskować wyłącznie na podstawie kluczy.
 - Struktura danych jest zróżnicowana lub dane mogą być niekompletne.

Typ ARRAY

- ARRAY w Snowflake działa podobnie jak tablica w wielu innych językach programowania.
- Tablica zawiera zero lub więcej elementów danych.
- Dostęp do każdego elementu uzyskuje się poprzez podanie jego pozycji w tablicy.

Typ ARRAY

- Każda wartość w tablicy częściowo ustrukturyzowanej jest typu VARIANT. Wartość typu VARIANT może zawierać wartość dowolnego innego typu danych.
- Wartości innych typów danych można rzutować na wartości typu VARIANT, a następnie zapisać w tablicy. Niektóre funkcje obsługujące tablice, w tym ARRAY_CONSTRUCT, mogą niejawnie rzutować wartości na wartości typu VARIANT.
- Ponieważ tablice przechowują wartości typu VARIANT, a wartości typu VARIANT mogą przechowywać w sobie inne typy danych, podstawowe typy danych wartości w tablicy mogą być różne.

Typ ARRAY

- W odniesieniu do tablic obowiązują następujące zasady:
 - Snowflake nie obsługuje tablic zawierających elementy określonego typu innego niż VARIANT.
 - Tablice w Snowflake deklaruje się bez określania liczby elementów. Tablice mogą dynamicznie powiększać się w wyniku operacji takich jak `ARRAY_APPEND`. Snowflake nie obsługuje obecnie tablic o stałym rozmiarze.
 - Tablica może zawierać zarówno wartości SQL NULL, jak i wartości null w formacie JSON. Więcej informacji można znaleźć w sekcji Wartości NULL.
 - Teoretyczny maksymalny łączny rozmiar wszystkich wartości w tablicy wynosi 128 MB.

Typ ARRAY

- Aby wstawić dane typu ARRAY bezpośrednio, należy użyć instrukcji INSERT INTO ... SELECT.

```
CREATE OR REPLACE TABLE array_example (array_column ARRAY);  
INSERT INTO array_example (array_column)  
  SELECT ARRAY_CONSTRUCT(12, 'twelve', NULL);
```

Typ ARRAY

- Snowflake obsługuje używanie stałych do określania wartości typu ARRAY. Stałe typu ARRAY są ograniczone nawiasami kwadratowymi ([i]).

```
[<value> [, <value> , ...]]
```

- VALUE - wartość przypisana do elementu tablicy. Może to być literał lub wyrażenie. Wartość ta może mieć dowolny typ danych.
- [] to pusta wartość typu ARRAY.
- [1, 'value1'] zawiera określone wartości stałej typu ARRAY, przy czym jako wartości użyto literałów.
- [c1+1, c1+2] zawiera określone wartości stałej typu ARRAY, przy czym jako wartości użyto wyrażeń.

Typ ARRAY

- W poniższym przykładzie użyto stałej typu ARRAY do określenia tablicy, którą należy wstawić.

```
INSERT INTO array_example (array_column)
SELECT [ 12, 'twelve', NULL ];
```

- W poniższych instrukcjach wykorzystano stałą typu ARRAY oraz funkcję ARRAY_CONSTRUCT do wykonania tego samego zadania:

```
UPDATE my_table SET my_array = [ 1, 2 ];

UPDATE my_table SET my_array = ARRAY_CONSTRUCT(1, 2);
```

- W poniższym przykładzie wykorzystano wyrażenia do określenia wartości w stałej typu ARRAY:

```
SET my_variable = 10;
SELECT [$my_variable+1, $my_variable+2];
```

```
+-----+
| [MY_VARIABLE+1, MY_VARIABLE+2] |
+-----+
| [                               |
|   11,                           |
|   12                             |
| ]                               |
+-----+
```

Typ ARRAY

- W instrukcjach SQL literały łańcuchów znaków w tablicach są zapisywane w pojedynczych cudzysłowach (podobnie jak w pozostałych częściach języka SQL w Snowflake), ale w wyświetlaniu literały łańcuchów znaków w tablicach są pokazywane w podwójnych cudzysłowach:

```
SELECT [ 'Alberta', 'Manitoba' ] AS province;
```

```
+-----+  
| PROVINCE |  
+-----+  
| [        |  
|  "Alberta", |  
|  "Manitoba" |  
| ]        |  
+-----+
```

Typ ARRAY

- Indeksy tablicy są liczone od 0, więc pierwszy element tablicy to element o indeksie 0.
- Dostęp do wartości w tablicy uzyskuje się poprzez podanie numeru indeksu elementu tablicy w nawiasach kwadratowych.
- Na przykład poniższe zapytanie odczytuje wartość znajdującą się pod indeksem 2 w tablicy zapisanej w kolumnie my_array_column.

```
SELECT my_array_column[2] FROM my_table;
```

- Tablice mogą być zagnieżdżone. Poniższe zapytanie odczytuje zerowy element zerowego elementu zagnieżdżonej tablicy:

```
SELECT my_array_column[0][0] FROM my_table;
```

Typ ARRAY

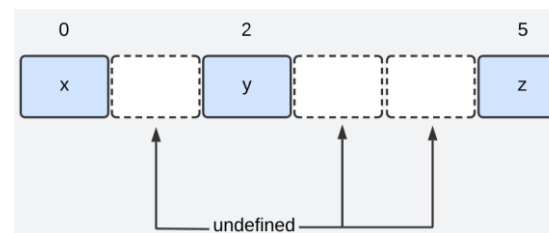
- Próba uzyskania dostępu do elementu znajdującego się poza końcem tablicy zwraca wartość NULL.
- Slice of array to sekwencja sąsiadujących ze sobą elementów (czyli ciągły podzbiór tablicy).
- Dostęp do slice of array można uzyskać poprzez wywołanie funkcji `ARRAY_SLICE`.
Na przykład:

```
SELECT ARRAY_SLICE(my_array_column, 5, 10) FROM my_table;
```

- Funkcja `ARRAY_SLICE` zwraca elementy od podanego elementu początkowego (w powyższym przykładzie 5) do podanego elementu końcowego (w powyższym przykładzie 10), nie włączając go.
- Pustą tablicę lub pusty wycinek często oznacza się parą nawiasów kwadratowych bez zawartości (`[]`).

Typ ARRAY – dense, sparse

- W tablicy gęstej indeksy elementów zaczynają się od zera i są uporządkowane sekwencyjnie (0, 1, 2 itd.). Natomiast w tablicy rzadkiej indeksy mogą być nieuporządkowane (na przykład 0, 2, 5). Wartości nie muszą zaczynać się od 0.
- Jeśli indeks nie ma odpowiadającego mu elementu, wówczas wartość odpowiadająca temu indeksowi jest określana jako niezdefiniowana. Na przykład, jeśli tablica rzadka ma trzy elementy, a elementy te znajdują się pod indeksami 0, 2 i 5, wówczas elementy pod indeksami 1, 3 i 4 są niezdefiniowane.



Typ ARRAY – dense, sparse

- Próba uzyskania dostępu do fragmentu wykraczającego poza koniec tablicy skutkuje zwróceniem pustej tablicy, a nie tablicy zawierającej wartości niezdefiniowane. Poniższe polecenie SELECT próbuje odczytać dane wykraczające poza ostatni element przykładowej tablicy rzadkiej:

```
SELECT ARRAY_SLICE(array_column, 6, 8) FROM table_1;
```

```
+-----+  
| array_slice(array_column, 6, 8) |  
+-----+  
| [ ]                               |  
+-----+
```

- Wartość „undefined” różni się od wartości NULL. Wartość NULL w tablicy jest elementem zdefiniowanym.
- W tablicy gęstej każdy element zajmuje miejsce w pamięci, nawet jeśli jego wartość wynosi NULL. W tablicy rzadkiej elementy o wartości „undefined” nie zajmują bezpośrednio miejsca w pamięci.

Typ ARRAY

- Dane typu ARRAY są zazwyczaj wykorzystywane, gdy spełniony jest co najmniej jeden z poniższych warunków:
 - Istnieje zbiór danych, a każdy element tego zbioru ma taką samą lub podobną strukturę.
 - Każdy element danych jest przetwarzany w podobny sposób. Można na przykład przechodzić cyklicznie przez dane, przetwarzając każdy element w ten sam sposób.
 - Dane mają naturalną kolejność, na przykład chronologiczną.

Typ ARRAY

```
CREATE OR REPLACE TABLE test_semi_structured(  
  var VARIANT,  
  arr ARRAY,  
  obj OBJECT);
```

```
CREATE OR REPLACE TABLE demonstration1 (  
  ID INTEGER,  
  array1 ARRAY,  
  variant1 VARIANT,  
  object1 OBJECT);  
  
INSERT INTO demonstration1 (id, array1, variant1, object1)  
SELECT  
  1,  
  ARRAY_CONSTRUCT(1, 2, 3),  
  PARSE_JSON(' { "key1": "value1", "key2": "value2" } '),  
  PARSE_JSON(' { "outer_key1": { "inner_key1A": "1a", "inner_key1B": "1b" }, '  
    ||  
    ' "outer_key2": { "inner_key2": 2 } } ');  
  
INSERT INTO demonstration1 (id, array1, variant1, object1)  
SELECT  
  2,  
  ARRAY_CONSTRUCT(1, 2, 3, NULL),  
  PARSE_JSON(' { "key1": "value1", "key2": NULL } '),  
  PARSE_JSON(' { "outer_key1": { "inner_key1A": "1a", "inner_key1B": NULL }, '  
    ||  
    ' "outer_key2": { "inner_key2": 2 } '  
    ||  
    ' } ');
```

```
SELECT *  
FROM demonstration1  
ORDER BY id;
```

ID	ARRAY1	VARIANT1	OBJECT1
1	[1, 2, 3]	{ "key1": "value1", "key2": "value2" }	{ "outer_key1": { "inner_key1A": "1a", "inner_key1B": "1b" }, "outer_key2": { "inner_key2": 2 } }
2	[1, 2, 3, NULL]	{ "key1": "value1", "key2": null }	{ "outer_key1": { "inner_key1A": "1a", "inner_key1B": null }, "outer_key2": { "inner_key2": 2 } }